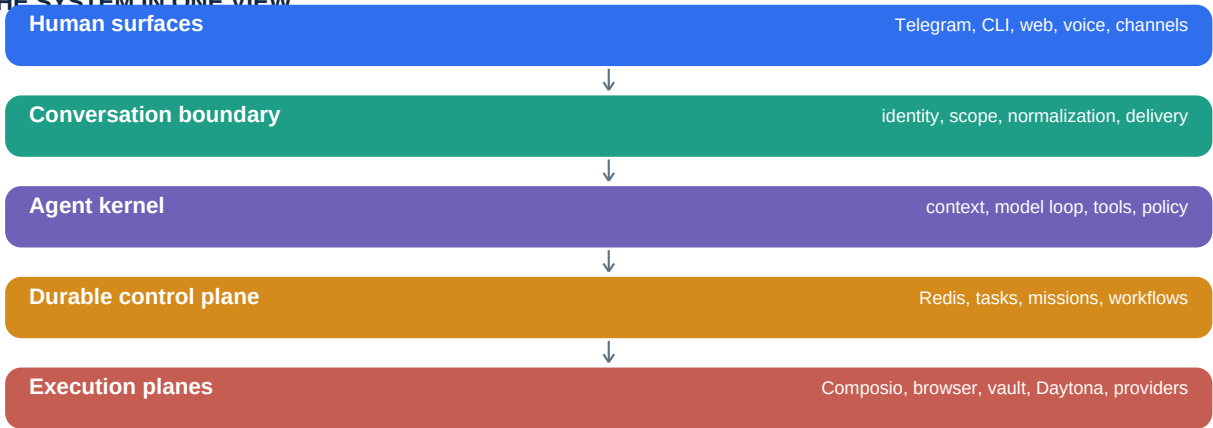


Chusky

The multilayered architecture and operating theory of a persistent, channel-spanning agent.

THE SYSTEM IN ONE VIEW



This document explains how the parts fit together, why the boundaries exist, and how Chusky turns natural language into durable, owner-scoped work. It is intentionally more theoretical than a deployment guide: the goal is to give engineers, operators, and product designers a shared mental model.

North star

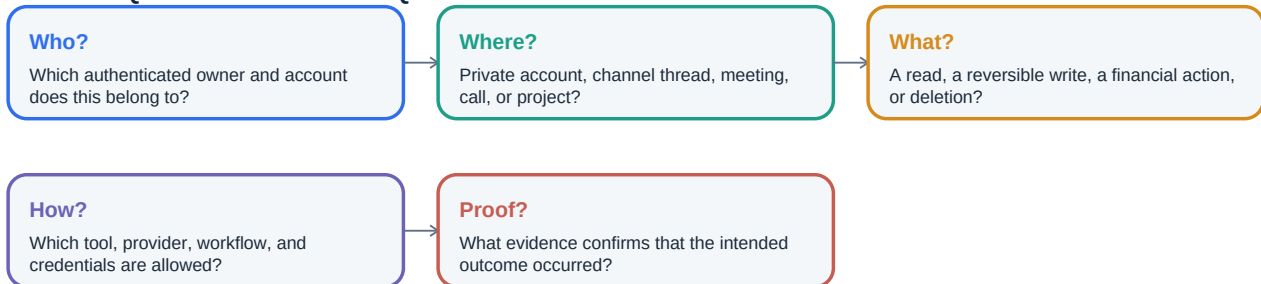
Chusky is not a collection of chat adapters. It is an owner-scoped operating system for intent: channels are the senses, the agent kernel is the reasoning core, native tools are the control surface, and durable state is the memory and accountability layer.

How to read this architecture

The design is easier to understand as a set of contracts than as a set of files.

The outer edge receives events from people and providers. The middle translates those events into a stable internal language. The agent kernel plans and acts using a bounded tool catalog. The durable substrate makes work resumable and auditable. External execution planes perform side effects only after ownership, policy, and verification checks.

THE FIVE QUESTIONS EVERY REQUEST MUST ANSWER



A model response is not the source of truth. It is a proposal. Chusky's runtime makes the proposal executable only when the proposal matches the authenticated owner, the current state, the exposed schema, and the policy for that action.

Contents

The document is organized from theory to implementation and then to operations.

Part	Focus	What it answers
01	System thesis	The core abstractions and why a layered agent is needed.
02	Runtime layers	The edge, agent kernel, control plane, and execution planes.
03	Channels and identity	Private continuity, shared scopes, and normalized provider events.
04	The agent loop	Model inference, tools, retries, modalities, and completion evidence.
05	Memory and context	History, summaries, structured memory, vector search, and context graphs.
06	Tools and policy	Native tools, Composio tools, approvals, and risk classification.
07	Durability	Tasks, waits, missions, leases, workflows, and recovery.
08	Browser, vault, and shopping	Private websites, credentials, checkout, and human handoffs.
09	Meetings and voice	Context-aware live interaction and low-latency voice theory.
10	Developer platform	REST, SDK, webhooks, MCP, A2A, and company workspaces.
11	Deployment and operations	Redis, workflow providers, releases, and incident response.
12	Testing and future theory	Evaluation, invariants, limits, and evolution.

As-built versus theory

The descriptions in this reference are grounded in the repository's architecture references, source layout, native-tool contracts, and test inventory. Where a concept is intentionally forward-looking, it is labeled as a design principle or extension point rather than a claim of live provider coverage.

01 / System thesis

A personal and business agent needs more than a chat loop.

A simple chatbot maps text to text. Chusky maps intent to outcomes. That difference introduces identity, state, policy, side effects, verification, and time. The system must know which person is speaking, which conversation scope is active, what the person has already authorized, what work is still running, and whether an outside system actually accepted the requested change.

The architecture therefore treats every request as a small distributed system problem. Natural language is the input language, but the result is a typed, owner-scoped, resumable operation. The agent can reason freely inside its prompt, but the runtime is the final authority for tools, credentials, ownership, approvals, and durable state.

Design law 1

Inference may suggest. Runtime boundaries decide.

Design law 2

Persistence preserves continuity, not permission. A memory that says a user wanted something yesterday does not authorize a financial action today.

Design law 3

A completed sentence is not a completed action. Completion requires provider evidence or a durable internal settlement.

The agent as an operating system

Chusky's value is the coordination of capabilities, not any single model.

The model is replaceable. The operating contracts are not. OpenRouter provides model access; Composio provides connected-app actions; E2B provides a retained browser; Daytona provides a private computer and workspace; Cloudflare provides a credential broker; Redis and workflow providers provide durable coordination. Chusky binds them into one owner-aware runtime.

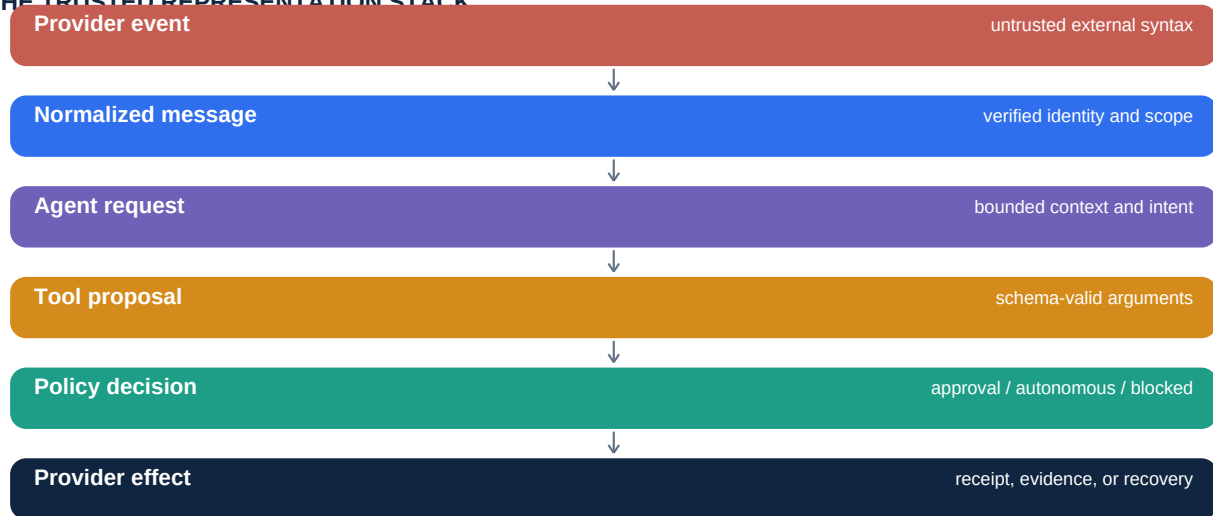
Abstraction	Question	Failure if missing
Identity	Who owns this request?	Cross-user data or action leakage
Scope	Which room or thread is active?	Private context appears in shared rooms
Intent	What outcome is requested?	The model optimizes the wrong task
Capability	Which tool can perform it?	Tool sprawl and unsafe dispatch
Policy	What level of authorization is needed?	Silent destructive or financial action
Durability	How does work survive time and retries?	Lost work or duplicate side effects
Evidence	What proves success?	Confident but false completion

This is why Chusky should be thought of as an intent operating system: the conversational surface is only the entry point into a graph of trusted state and controlled execution.

02 / Runtime layers

The architecture is a stack of increasingly trusted representations.

THE TRUSTED REPRESENTATION STACK



Every transition should reduce ambiguity. A provider payload may contain dozens of fields; the normalized message contains only the safe information required for the agent. A model tool call may contain arbitrary JSON; the runtime accepts only the exact schema and ownership rules. A provider response may be verbose or malicious; the runtime keeps only bounded evidence.

Layer A: human and provider surfaces

Channels are ingress and egress adapters, not separate brains.

Telegram, CLI, web, Slack, WhatsApp, Sendblue iMessage, SMS, voice, meetings, and developer APIs are different transport surfaces over the same agent and persistence contracts. Their job is to authenticate or verify the event, normalize it, invoke the shared handler, and render the result safely for that provider.

Surface	Primary responsibility	Do not put here
Telegram	Commands, callbacks, media orchestration, rendering	Provider-neutral tool logic
CLI	Pairing, terminal input, shared session client	OpenRouter or Composio secrets
Web	Authenticated dashboard and API client	Server secrets in browser code
Channels	Signature checks, normalization, outbox delivery	Raw provider payloads in context
Voice	Audio transport, turn timing, speech events	Unbounded private data
SDK/MCP	Developer and company integration contracts	Bypassing ownership and policy

The surface does not decide whether a purchase is safe. It reports the user's intent and displays the runtime's decision. This keeps parity across channels without copying business logic into every adapter.

Layer B: the conversation boundary

The boundary converts transport identity into Chusky identity.

The channel gateway resolves a verified provider identity to an internal account key such as `account_`. It then chooses a private account conversation or an explicitly authorized shared conversation. This is the point where the system decides whether private history, private memory, tasks, approvals, and connected accounts are even eligible for loading.

PRIVATE AND SHARED-CONTEXT

Verified identity

Provider signature, link code, OAuth result, or paired device proves the external identity.



Conversation scope

Private DMs load account context; shared rooms load only room-scoped history and grants.



Agent context

Only relevant, bounded context is injected; raw provider payloads never become authority.



Delivery

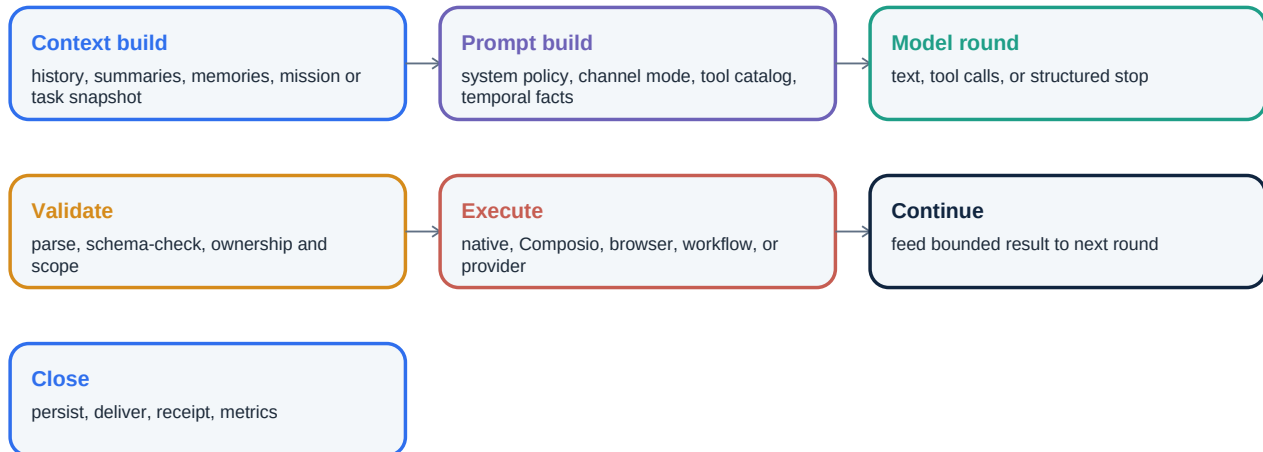
Outbound content returns through the same owner or shared-room boundary and durable outbox.

Layer C: the agent kernel

One loop, many capabilities.

src/agent.ts owns the model inference loop, Composio session reuse, modality routing, retry behavior, tool execution, and approval boundaries. It should not know Telegram formatting or Redis key details. It receives a normalized request and returns a bounded result or a durable pause state.

THE SHARED AGENT KERNEL



03 / Channels and identity

Continuity is shared at the account boundary; transcripts remain scope-specific.

The same owner can speak from Telegram, CLI, the web dashboard, iMessage, Slack, WhatsApp, SMS, a phone call, or a meeting. The account identity gives the agent continuity, but it does not mean every surface should display every transcript. Shared history is context for the agent, not a cross-channel transcript UI.

State	Private surface	Shared surface
Recent history	Owner account session	Provider/thread record
Structured memory	Eligible when relevant	Never inherit account-only memory
Connected accounts	Owner-scoped	Only exact shared grants
Approvals	Owner and exact arguments	Room/owner policy as designed
Tasks and missions	Owner-scoped continuation	Only linked public work
Delivery	Direct channel	Authorized thread or group

This split lets Chusky feel like one agent to a person without turning a group room into a window into that person's private life.

Normalization and durable delivery

A message is not complete until the delivery protocol settles it.

Adapters verify raw requests against provider-specific signatures or secrets, deduplicate event IDs, normalize text and media, and create an internal inbound event. The channel gateway claims the event, invokes the shared agent handler, and persists the outbound response in a leased outbox before sending. Provider receipts and retry state remain durable.

- 1 Verify the raw request before parsing provider fields.
- 2 Resolve the external identity to exactly one Chusky owner or authorized shared scope.
- 3 Claim the provider event ID so retries cannot create duplicate work.
- 4 Normalize safe text, media references, thread identifiers, and capability metadata.
- 5 Run the common agent path under the owner or conversation lock.
- 6 Persist the response as an outbox record before delivery.
- 7 Send, record the provider receipt, and release the lease idempotently.

CLI and web are clients, not alternate agents

Parity comes from reusing the server lifecycle.

The CLI pairs a device to an owner, stores only a token hash server-side, and calls `/cli/chat` or `/cli/chat/stream`. It shares the owner session and lock with Telegram. The web dashboard calls the authenticated `/v1` API and renders selected-thread history; it should not merge a separate account-history feed into the visible transcript.

The important theory is that transport parity is a consequence of server-side convergence. If CLI and web each grew their own model loop, memory logic, or approval logic, they would eventually disagree about what the agent knows and what it is allowed to do. The correct extension point is the shared agent/store boundary.

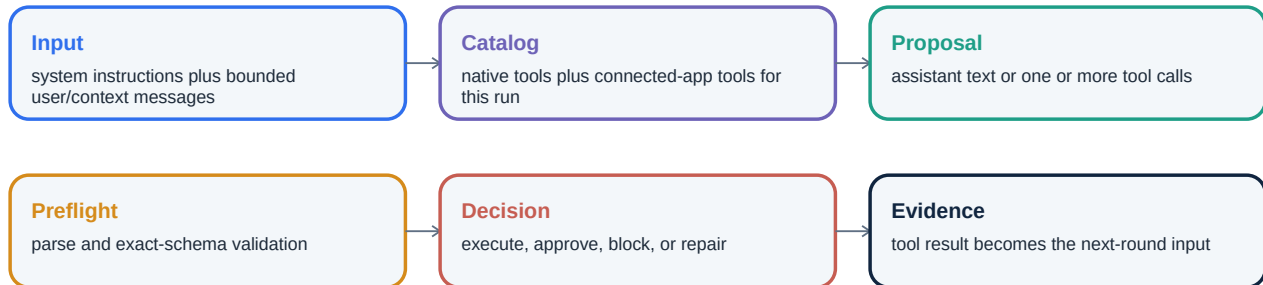
Operational invariant

A new channel should be a new adapter and renderer, not a new copy of Chusky's intelligence.

04 / The agent loop

Inference is a bounded conversation with the runtime, not a single API call.

A MODEL ROUND



The loop continues until the model returns a final response, a durable wait, an approval pause, or the configured round budget is exhausted. Each tool call is independently checked. A tool call that is malformed, length-limited, schema-invalid, unauthorized, or outside the current session must not consume the same execution path as a valid call.

Prompting as a control surface

The prompt explains behavior; the runtime enforces it.

System prompts should tell the agent how to speak, when to use a tool, what a successful outcome looks like, and how to handle uncertainty. Voice and meeting prompts add turn-taking, acknowledgments, short spoken responses, and tool-progress language. However, prompts cannot be the only security mechanism: a prompt saying 'do not delete' does not replace a deletion guard.

Prompt responsibility	Runtime responsibility
Explain channel mode and audience	Enforce conversation scope
Describe available capabilities	Expose only current schemas
Teach natural tool narration	Prevent unverified completion claims
Suggest when to ask or pause	Persist approvals and waits
Give meeting or voice examples	Bound latency, audio, and turn state
Explain memory relevance	Filter owner and retention boundaries

Modality, retries, and completion

The loop must preserve truth under imperfect providers.

For images, documents, audio, and video, Chusky checks the selected model's input capabilities and uses a configured fallback only when necessary. The selected model remains the user's choice; temporary media routing must not silently change the persistent model. OpenRouter or provider retries are bounded and should occur only before partial output reaches the client for streamed responses.

Completion rule

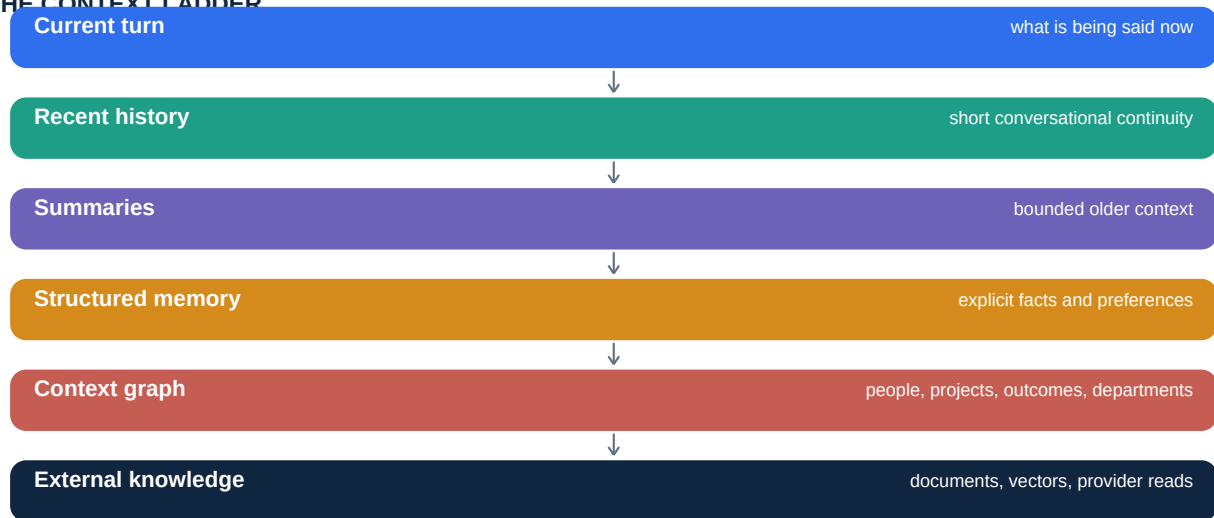
The agent can say 'I am checking' before a tool call. It may say 'done' only after the tool result, provider receipt, or durable internal record supports that claim.

Tool results are untrusted data. They can contain prompt injection, misleading instructions, or oversized payloads. The runtime bounds them, labels them as results, and keeps authorization in the server's policy context.

05 / Memory and context

Persistence is layered so intelligence remains useful without becoming uncontrolled.

THE CONTEXT LADDER



The agent should not treat all memory as equal. Current-turn content is immediate but ephemeral. Recent history preserves phrasing and local intent. Summaries compress old conversations. Structured memories require explicit saving or a clearly durable preference. Context graphs connect projects and relationships. External knowledge is retrieved for a purpose and must remain bounded.

History, summaries, and Redis pressure

Bounded persistence is a reliability feature, not an intelligence reduction.

A session record contains model choice, history, summaries, memories, scratchpad, approvals, reminders, jobs, trigger IDs, Composio session identity, usage, and timestamps. The store must normalize old records, cap history, and preserve important durable entities separately from the rolling transcript.

Layer	Retention strategy	Why it matters
Raw history	Bound recent turns; summarize overflow	Natural continuity without unbounded payloads
Summaries	Keep a small bounded tail	Long-range topic continuity
Memory facts	Explicit category, scope, expiry, confidence	Reliable personalization
Scratchpad	Private temporary notes with clear lifecycle	Working state without permanent memory
Durable records	Separate keys and schemas	Tasks and approvals survive session writes
Vector index	Optional semantic retrieval with fallback	Relevance search, not authority

When Redis is under pressure, the correct response is to reduce repeated read-modify-write amplification and bound large session values. It is not to delete intelligence indiscriminately or make production silently fall back to memory.

Context graph and bounded retrieval

Relevance is a policy decision.

Context nodes represent people, projects, departments, observations, outcomes, and relationships. Links are owner-scoped and bounded. A query should retrieve only the context relevant to the current objective, with safe labels and confidence rather than injecting the entire account history.

- 1 Identify the active owner and conversation scope.
- 2 Extract the current objective and entities involved.
- 3 Search structured memory and context nodes with narrow filters.
- 4 Optionally search vectors, while treating results as recall candidates.
- 5 Rank and truncate context to a prompt budget.
- 6 Label evidence, inference, and uncertainty separately.
- 7 Do not persist new long-term facts unless the save contract is satisfied.

06 / Tools and policy

Capabilities are explicit contracts with server-side enforcement.

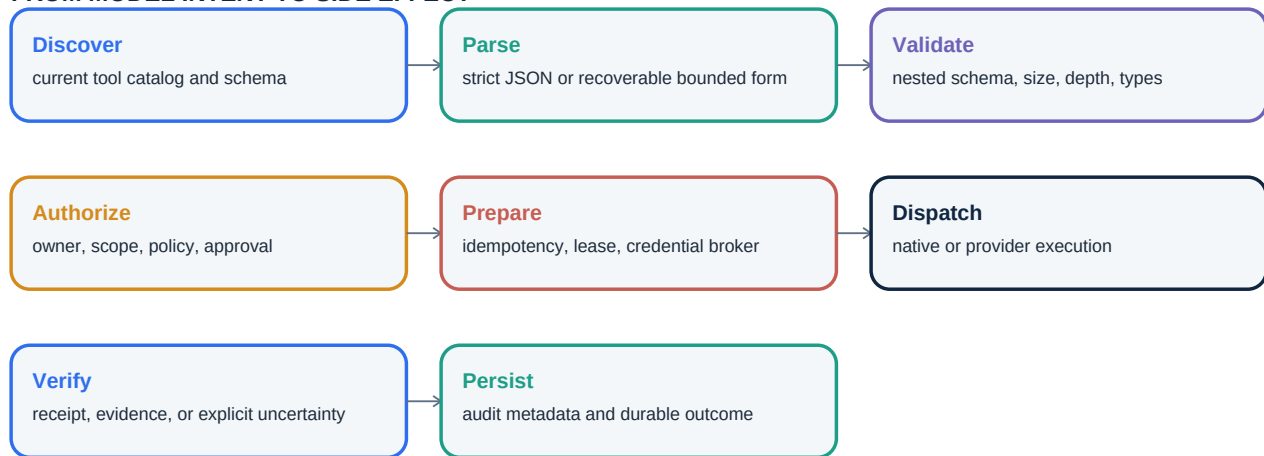
Native tools are model-facing schemas for Chusky-owned capabilities. Composio tools are connected-app capabilities discovered for the current session. Browser, vault, Daytona, Treg, Link, missions, tasks, and channel operations all become tools only through their own policy and ownership boundary.

Tool family	Examples	Primary proof
Read-only	search, list, inspect, health	Observed provider or durable state
Reversible write	message, note, calendar update	Provider receipt and idempotency
External high-impact	purchase, payment, delete, permission change	Exact approval plus provider settlement
Durable control	task wait, mission pause, reminder	State transition and owner CAS
Execution bridge	browser, file, artifact, media	Verified result and artifact evidence

Tool lifecycle

Every tool call passes through the same conceptual gates.

FROM MODEL INTENT TO SIDE EFFECT

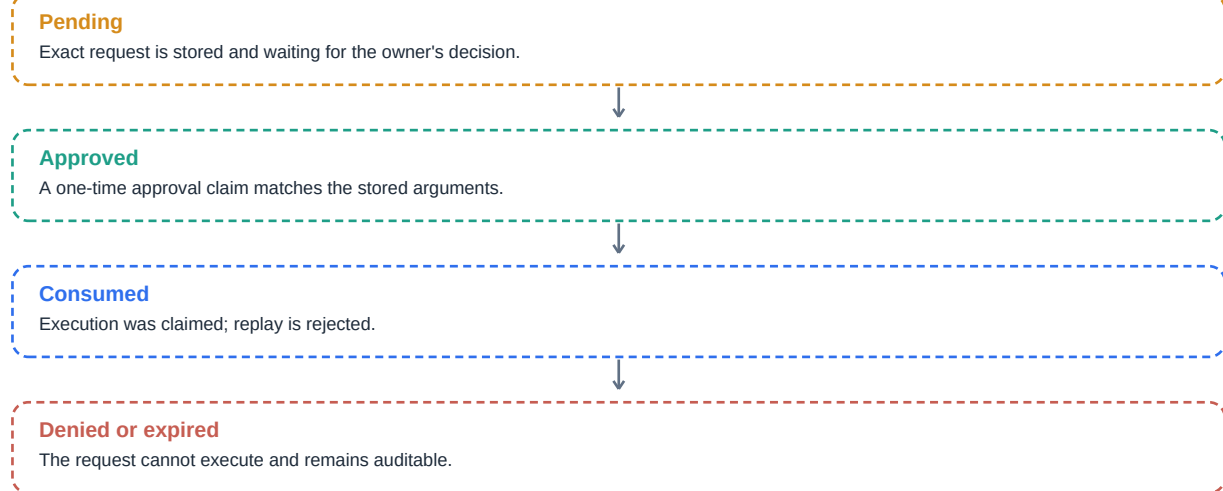


Approvals are state, not UI

A button is only the visible part of an approval protocol.

When approval is required, Chusky persists the user ID, tool slug, exact serialized arguments, original request, model, expiry, and pending status. The user sees a concise review prompt. Approval is claimed atomically, matched against the original arguments, consumed once, and rejected if expired, denied, foreign, or changed.

APPROVAL STATE MACHINE



07 / Durability

Time, retries, and multiple workers are normal operating conditions.

A durable task is not a background promise. It is a record with an owner, objective, status, attempt budget, checkpoint, exact next action, wake-up time, workflow ID, lease information, and settlement rules. A mission composes tasks into dependency-aware steps and adds evidence, verification, repair, and replanning.

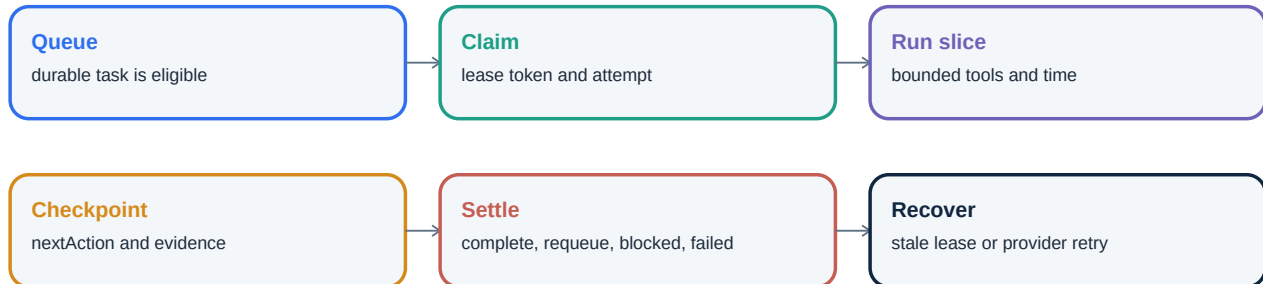
Primitive	Use it for	Key invariant
Reminder	One future notification	Delivery claims are idempotent
Recurring job	Repeated schedule	Occurrences do not mutate the job contract
Task	Bounded durable work	One lease holder settles the attempt
Mission	Multi-step objective	Dependencies and proof control completion
Attention pulse	Proactive reconciliation	Deduplicated owner-scoped signal
Workflow	Delayed or retried execution	Payload is minimal and replay-safe

Leases, CAS, and recovery

Distributed correctness comes from ownership tokens and version checks.

Multiple replicas may see the same queued record. A worker claims it with a lease token and attempt number. Mutations use compare-and-set or equivalent version checks so concurrent updates preserve both changes or cause one worker to retry. Settlement verifies the same lease token; a stale worker cannot mark another worker's task complete.

WORKER EXECUTION



Missions as a theory of work

A mission is a dependency graph with proof obligations.

A mission begins with an objective and a bounded budget. The scheduler materializes dependency-ready steps as durable tasks. Each step produces a checkpoint and evidence. The mission is not complete merely because a model says it is complete; required evidence and verification must settle first.

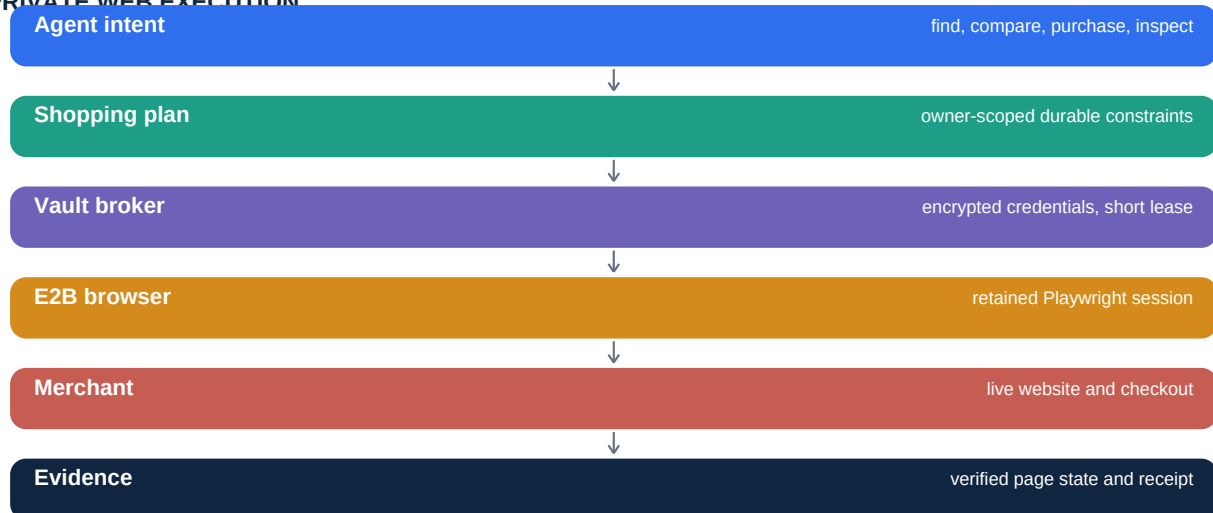
- 1 Parse the objective into typed steps and dependencies.
- 2 Reject semantically invalid routing before persisting the mission.
- 3 Schedule only dependency-ready work.
- 4 Execute each step in a bounded slice with a named capability contract.
- 5 Wait for exact external events rather than spinning.
- 6 Re-read mission state before every autonomous continuation.
- 7 Verify the definition of done and preserve evidence.
- 8 Repair or replan only through explicit state transitions.

08 / Browser, vault, and shopping

Web automation is powerful because the browser boundary is explicit.

Chusky separates three concerns: automated browser interaction in E2B, desktop/file/application work in Daytona, and credential custody in the Cloudflare vault broker. Composio remains the preferred path for OAuth-connected apps. The vault is for ordinary websites whose login must be performed through a browser.

PRIVATE WEB EXECUTION



The browser operating loop

Observe before acting; verify after acting.

- 1 Open only an explicit safe HTTP(S) origin.
- 2 Inspect the live accessibility tree or page snapshot.
- 3 Generate bounded candidate actions from observed controls.
- 4 Prefer accessible node actions over coordinates.
- 5 Perform one consequential action at a time.
- 6 Re-inspect after navigation, form submission, or checkout mutation.
- 7 Pause for CAPTCHA, MFA, passkeys, age verification, or site-only challenge.
- 8 Return only bounded state and evidence; never export cookies or credentials.

Why the loop matters

A model-authored selector is not evidence that a button exists. The browser must observe the page and the runtime must verify that the resulting state matches the intended action.

Shopping and checkout theory

Cart autonomy and payment authority are different levels.

Stage	Agent may do	Runtime rule
Discover	Search products and compare options	Current availability must be observed
Plan	Create a private shopping run	Budget and substitutions are explicit
Cart	Add, remove, and update items	Cart-building is reversible
Checkout	Review delivery, taxes, and total	High-impact boundary begins
Payment	Prepare approved payment mechanism	Financial approval required
Order	Submit exactly once	Provider receipt and idempotency
Aftercare	Track receipt, delivery, refund	Durable outcome record

This lets Chusky act like a capable shopping assistant without confusing browsing authority with payment authority. A user can ask it to build a cart autonomously and still retain control over the exact amount at checkout.

09 / Meetings and voice

Live conversation is a latency-sensitive channel with special context rules.

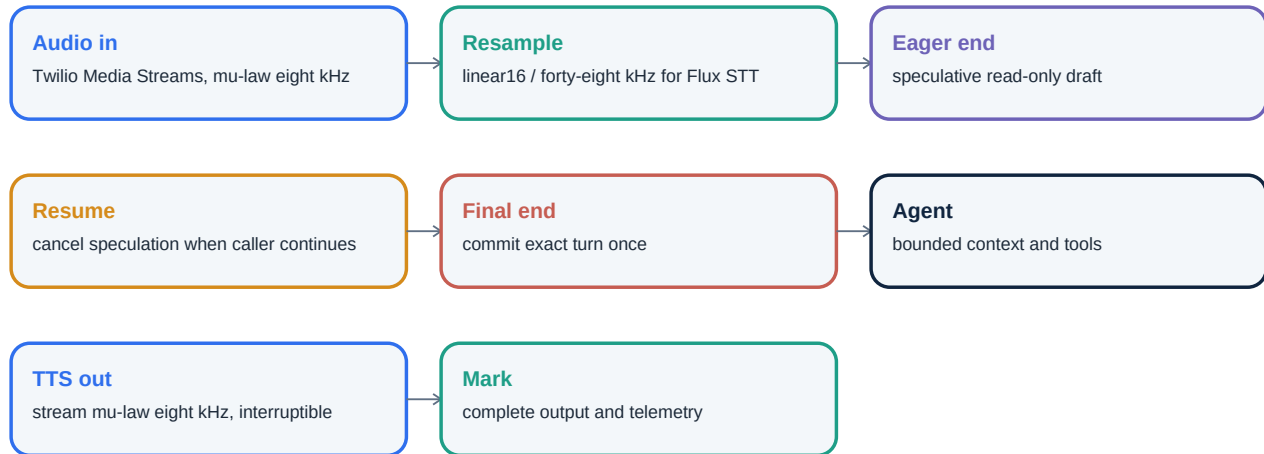
A meeting representative or phone agent is still the same owner-scoped Chusky, but the surface changes the operating contract. It must know it is in a meeting or call, speak in short turns, preserve turn-taking, avoid reading private unrelated facts into a shared room, and use read-only lookups smoothly while deferring high-impact actions to the owner's direct channel.

Voice requirement	Architectural response
Low latency	Streaming STT, eager turn speculation, bounded queues
Natural back-to-back turns	Commit only final end-of-turn, interrupt TTS on resumed speech
Meeting context	Inject meeting mode and relevant owner/business context
Tool narration	Prompt teaches short progress language; runtime dispatches tool
Privacy	Owner-private context for private calls; shared rooms stay scoped
High-impact actions	Read-only voice tools; approval continues in direct channel

The low-latency voice path

Audio is a streaming protocol, not a series of uploaded recordings.

VOICE TURN TIMELINE



Voice as a conversational theory

The system must sound connected without inventing certainty.

Voice prompts should forbid markdown, emoji, brackets, and special formatting because every token is spoken. They should encourage short acknowledgments, one question at a time, natural number pronunciation, and brief progress phrases before read-only lookups. The prompt should include bad and good examples so the model learns the shape of a natural exchange.

But conversational warmth must not become fake completion. 'Let me check that' is a useful bridge into a tool call. 'It is done' requires the tool result. A voice agent is trustworthy when its timing feels human and its claims remain evidence-based.

10 / Developer platform

REST, SDK, webhooks, MCP, and A2A expose the same control plane at different heights.

Surface	Best for	Core boundary
REST /v1	Direct application integration	Project key, owner/project isolation
SDK	Typed developer experience	Published types match authenticated API
Webhooks	Durable external events	Signature, dedupe, enqueue before ack
Remote MCP	Company agents and tool hosts	Project-scoped key and tool scopes
A2A	Agent-to-agent objectives	Typed tasks, bounded handoffs, no hidden authority
Native tools	Chusky's own agent loop	Current run catalog and runtime policy

These are not six independent products. They are six ways of entering the same owner and company control plane. The same durable task, approval, mission, or outcome should remain inspectable regardless of whether it began in a chat, an SDK call, an MCP tool, or an A2A handoff.

SDK and API theory

Public contracts should make the safe path the easy path.

The Developer API owns authenticated projects, threads, runs, approvals, tasks, files, audit events, and webhook subscriptions. Credentials are project-scoped and must never be confused with the root server key. Uploaded files are private and must not become unverified public downloads.

- 1 Authenticate the developer request and resolve the project.
- 2 Validate the owner, thread, run, and resource relationship.
- 3 Create or reuse a durable run with bounded input and model settings.
- 4 Expose approval and task state as first-class resources.
- 5 Deliver events through signed, idempotent webhooks.
- 6 Keep SDK types, OpenAPI, tests, changelog, and published package aligned.

Release law

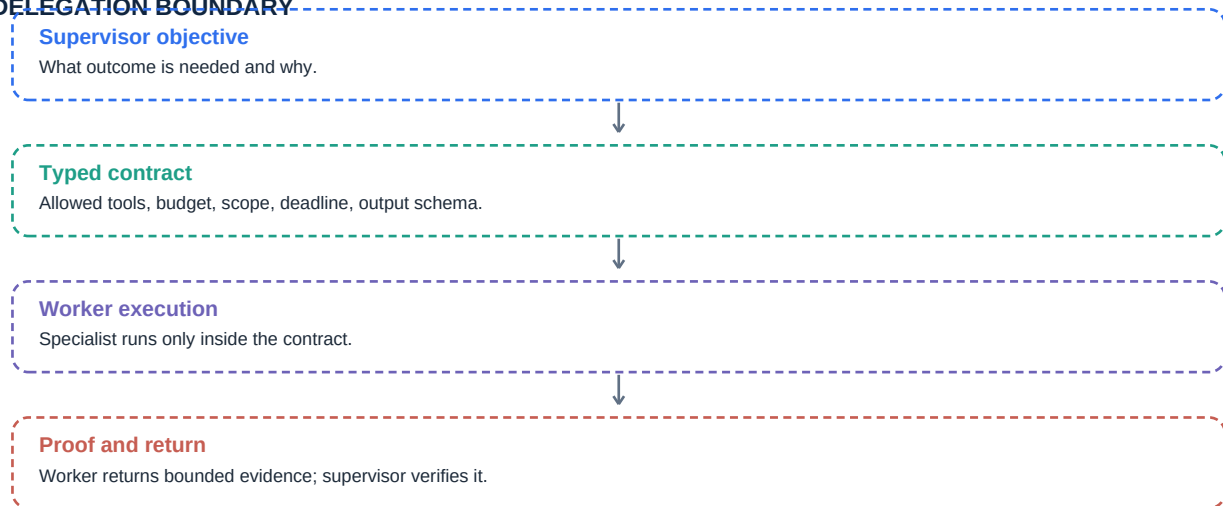
A public SDK feature is not complete when source code compiles. It is complete when the authenticated API, SDK types, package contents, changelog, registry version, and release tag agree.

Remote MCP and A2A

Delegation should carry objectives and contracts, not unrestricted power.

The remote MCP server is a company-facing integration boundary. It should expose discovery, durable execution, approvals, task operations, and company-level views without leaking private account data. A2A extends the same idea across agents: a supervisor can delegate a typed objective to a specialist, but the specialist receives a scoped contract, budget, allowed capability family, and expected result shape.

DELEGATION BOUNDARY



11 / Deployment and operations

Production is a distributed runtime, not a single Node process.

PRODUCTION DEPLOYMENT PLANE



Polling is appropriate for local Telegram development. External webhooks require a public HTTPS URL and Redis. Durable reminders, jobs, and long-running work require workflow configuration. The production path should fail closed when Redis is missing rather than silently degrade into in-memory state.

Configuration as architecture

Environment variables are a map of dependencies.

Domain	Representative configuration	Failure if absent
Model	OPENROUTER_API_KEY, model IDs	No inference or wrong modality
Connected apps	COMPOSIO_API_KEY	No external app actions
Persistence	REDIS_URL	Lost sessions and non-durable work
Workflows	QSTASH_TOKEN, public workflow URLs	Reminders and tasks do not resume
Browser	E2B flags and API key	No retained browser session
Vault	Cloudflare broker URL and HMAC	No secure website login
Voice	Twilio and Deepgram settings	No low-latency calls
Developer API	project/auth configuration	No external application access

Configuration is not merely deployment plumbing. It selects which execution planes exist and which failure modes are possible. A feature flag must produce an explicit unavailable response, not a half-working path that claims success.

Observability and incident response

The logs should explain the state machine without leaking the state payload.

Useful operational metadata includes owner hash, request or run ID, tool slug, approval ID, workflow ID, provider event ID, attempt number, duration, status, failure class, and receipt reference. Raw prompts, credentials, cookies, media bytes, full provider payloads, and unredacted arguments do not belong in logs.

Incident	First questions	Safe response
Redis outage	Which durable operations are affected?	Fail closed; preserve user-visible uncertainty
Provider timeout	Was the write outcome certain?	Do not blindly replay uncertain writes
Duplicate action	Which idempotency and receipt keys exist?	Quarantine and reconcile
Workflow failure	Was the task claimed or settled?	Recover stale lease, then retry bounded work
Model failure	Did partial output reach user?	Do not replay streamed partial answer
Credential issue	Is the identity expired or revoked?	Pause and request private re-authentication

12 / Testing and future theory

The test matrix is a model of the architecture.

The repository has tests for agent contracts, native tool schemas, policy, channels, webhooks, stores, memories, browser security, vault boundaries, shopping, missions, tasks, voice, meetings, SDK APIs, MCP clients, provider adapters, artifacts, media, and release notices. This breadth is important because an agent's failures usually cross module boundaries.

Test layer	What it proves	What it cannot prove alone
Unit	Pure validation and transformation	Provider availability
Store	Ownership, CAS, lifecycle, retention	Live Redis quota
Agent contract	Tool loop and completion semantics	Real merchant outcome
Handler/channel	Signature, normalization, delivery	Provider account configuration
Workflow	Replay, leases, retries, waits	Production queue health
Live smoke	Specific provider deployment path	All regions and tenants

A green suite is necessary but not sufficient. Production readiness also requires live configuration, provider verification, capacity monitoring, and an incident path that has been rehearsed.

Invariants worth defending

These are the system's constitutional rules.

- 1 No owner, no private context.
- 2 No verified scope, no shared-room access.
- 3 No current schema, no tool execution.
- 4 No exact approval match, no high-impact action.
- 5 No provider receipt, no external completion claim.
- 6 No lease ownership, no durable settlement.
- 7 No bounded payload, no persistence or dispatch.
- 8 No trusted vault path, no credential injection.
- 9 No evidence, no mission completion.
- 10 No release alignment, no public SDK claim.

These invariants are stronger than any individual model prompt. They are the difference between an agent that merely talks about work and an agent that can be trusted to do work over time.

What Chusky can become

A theory of capability without losing the human at the center.

The long-term opportunity is not to expose every tool in every conversation. It is to give the agent a broad internal world model and a durable action surface while preserving a small number of non-negotiable boundaries: ownership, shared-room privacy, deletion and financial control, credential custody, bounded execution, and evidence.

That approach supports a personal assistant, a business employee, a meeting representative, a voice operator, a developer agent, and a company specialist without creating separate brains. The surface changes the prompt, latency budget, and delivery shape. The core identity, memory, tools, policy, and durable outcome system remains one coherent operating model.

Final design statement

Chusky should be capable enough to carry an owner's work forward, humble enough to distinguish evidence from inference, and structured enough to recover when the outside world is slow, ambiguous, or wrong.

Appendix A / Repository map

The following map connects the theory to the source tree.

Area	Primary source locations	Architectural role
Agent loop	src/agent.ts, src/agentTools.ts	Inference, tools, retries, modalities
Persistence	src/store.ts, src/session.ts	Sessions, history, records, locks
Channels	src/channels/, src/handlers.ts	Normalization, delivery, provider contracts
Durability	src/taskRunner.ts, src/missionScheduler.ts, src/workflows/	Tasks, missions, delayed work
Context	src/contextGraph.ts, src/lib/knowledge/	Structured and semantic retrieval
Browser	src/vault/, src/lib/e2b/, e2b/	Retained browser and secure credentials
Shopping	src/shopping/	Retailer-neutral plan and checkout policy
Voice	chusky-voice/, src/voice*.ts, src/twilio*.ts	Streaming calls and meeting voice
Developer API	src/sdkApi.ts, sdk/, docs/a2a.md	REST, SDK, webhooks, A2A
MCP	cloudflare/, src/companyPlatform.ts	Company integration boundary
Testing	tests/	Contract, security, lifecycle, and provider seams

Appendix B / Source references

Repository-grounded references used to shape this document.

- `.agents/skills/chusky/references/architecture.md` - request lifecycle, module contracts, state transitions.
- `.agents/skills/chusky/references/security.md` - ownership, approvals, webhooks, replay, secret hygiene.
- `.agents/skills/chusky/references/browser.md` - E2B, Daytona, vault broker, Cloudflare Worker, browser verification.
- `.agents/skills/chusky/references/shopping.md` - shopping plans, retailer neutrality, checkout policy, handoffs.
- `.agents/skills/chusky/references/missions.md` - dependency scheduling, checkpoints, evidence, repair, replanning.
- `.agents/skills/chusky/references/durable-tasks.md` - tasks, reminders, recurring jobs, attention pulse, recovery.
- `.agents/skills/chusky/references/sdk/developer-api.md` - API boundary, files, webhooks, SDK releases.
- `.agents/skills/chusky/references/mcp.md` - remote MCP source, scopes, lifecycle, approvals, telemetry.
- `.agents/skills/chusky/references/deployment.md` and `operations.md` - runtime modes, production checks, incidents.
- `.agents/skills/chusky/references/testing.md` - required test layers and evaluation cases.
- `README.md` - user-facing capabilities, deployment surfaces, integrations, and architecture summary.
- `package.json` and `tests/` - executable project surface and current test inventory.

Scope note

This is an architecture reference, not a guarantee that every external provider is enabled in every deployment. Live provider behavior, account eligibility, quotas, credentials, and regional availability remain operational gates.

Appendix C / Review checklist

Use this when adding a capability or changing a boundary.

- 1 Which owner and conversation scopes can reach this capability?
- 2 What is the normalized input and what provider data is discarded?
- 3 Which native schema and implementation branch own the capability?
- 4 What validation happens before any external dispatch?
- 5 Is the action autonomous, approval-bound, or blocked? Why?
- 6 What is the idempotency key and what happens after an uncertain response?
- 7 What durable record or checkpoint makes the work resumable?
- 8 What evidence is required before the agent reports completion?
- 9 How are credentials injected without reaching the model?
- 10 Which tests cover valid, invalid, duplicate, unauthorized, retry, and provider-failure paths?
- 11 Which README, upgrade notice, SDK, OpenAPI, MCP, or deployment docs must change?
- 12 What live smoke test proves the configured production path?

End of reference.